

Salesforce Developer Bridge Program

Day 4 Task Submission Report

Submitted By: Midasala Nikshitha Ramesh [23PA1A05F2]

Project Title:

Hospital OPD Management System

Task 1: Creation and Deployment of Lightning Web Component

Objective

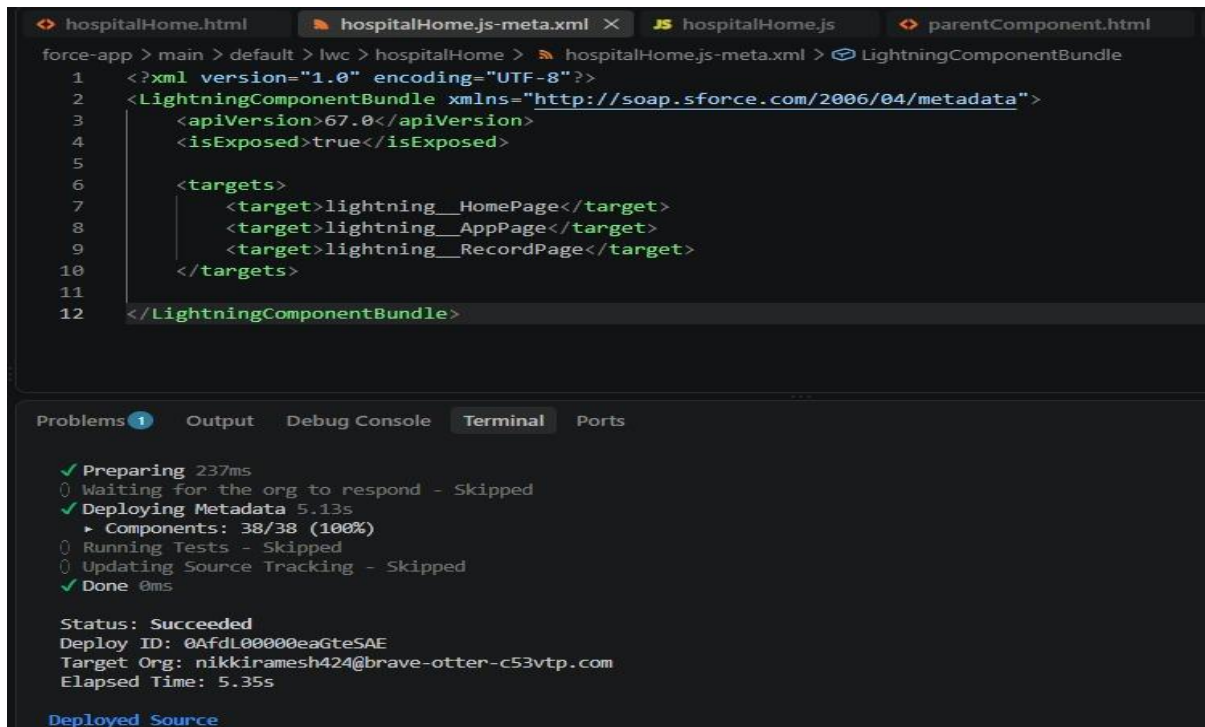
To create a custom Lightning Web Component (LWC), configure its metadata file, and deploy it successfully to the Salesforce Developer Organization for use on Lightning pages.

Work Performed

- Opened the Salesforce DX project in Visual Studio Code.
- Created a new Lightning Web Component named **Hospital OPD**.
- Configured the hospitalHome.js-meta.xml file by enabling the component and specifying the supported Lightning page targets such as Home Page, App Page, and Record Page.
- Verified the component configuration and deployed it to the connected Salesforce Developer Organization using Salesforce CLI.
- Monitored the deployment process and confirmed that all component files were successfully deployed without any errors.

Outcome

The Lightning Web Component was successfully created and deployed to the Salesforce Developer Organization. The successful deployment confirmed that the component was correctly configured and ready to be added to Lightning pages for further development.



```
force-app > main > default > lwc > hospitalHome > hospitalHome.js-meta.xml > LightningComponentBundle
1  <?xml version="1.0" encoding="UTF-8"?>
2  <LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
3      <apiVersion>67.0</apiVersion>
4      <isExposed>true</isExposed>
5
6      <targets>
7          <target>lightning__HomePage</target>
8          <target>lightning__AppPage</target>
9          <target>lightning__RecordPage</target>
10     </targets>
11
12 </LightningComponentBundle>
```

Problems 1 Output Debug Console Terminal Ports

✓ Preparing 237ms
⌚ Waiting for the org to respond - Skipped
✓ Deploying Metadata 5.13s
 ▶ Components: 38/38 (100%)
⌚ Running Tests - Skipped
⌚ Updating Source Tracking - Skipped
✓ Done 0ms

Status: Succeeded
Deploy ID: 0AfdL00000eaGteSAE
Target Org: nikkiramesh424@brave-otter-c53vtp.com
Elapsed Time: 5.35s

Deployed Source

Task 2: Deployment of the Component Using Lightning App Builder

Objective

To deploy the custom Lightning Web Component to the Salesforce Home Page using Lightning App Builder and verify its successful execution.

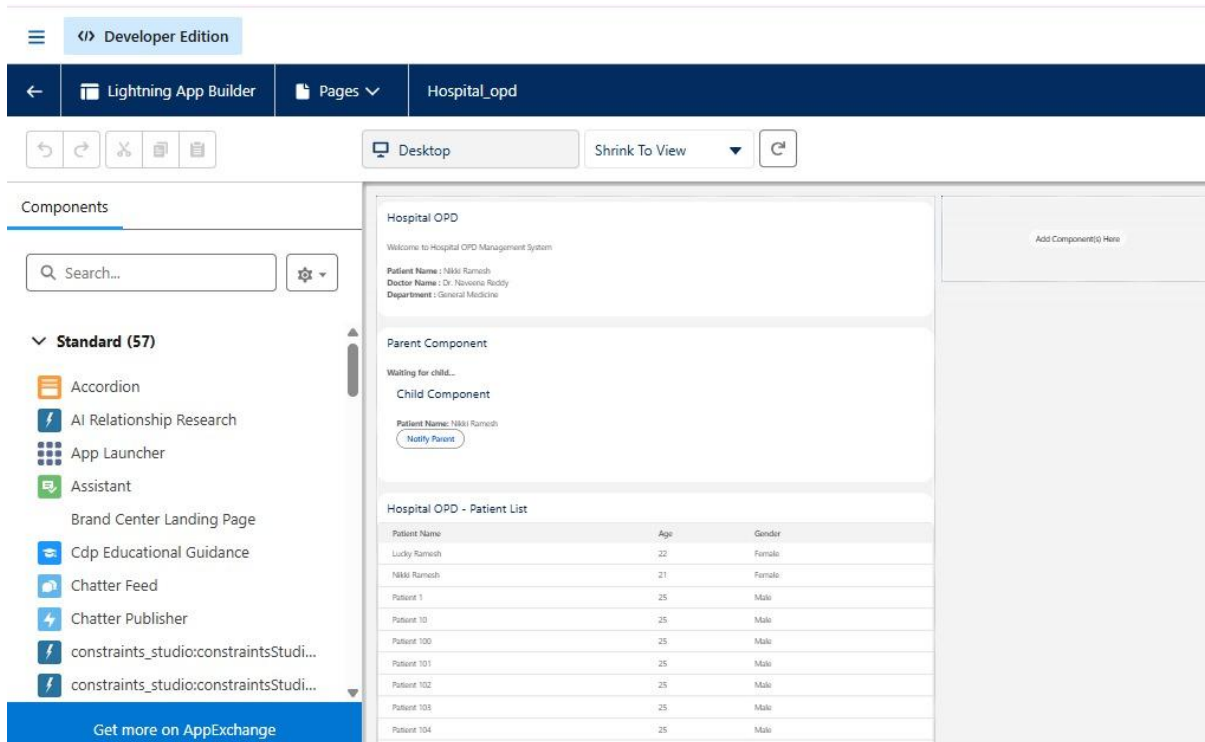
Work Performed

- Opened **Lightning App Builder** in Salesforce.
- Located the custom Hospital OPD component in the component palette.
- Dragged and dropped the component onto the Home Page layout.
- Saved and activated the Lightning page configuration.
- Opened the Salesforce Home Page to verify that the deployed component displayed correctly.
- Confirmed that the patient information, parent-child components, and patient list were rendered successfully.

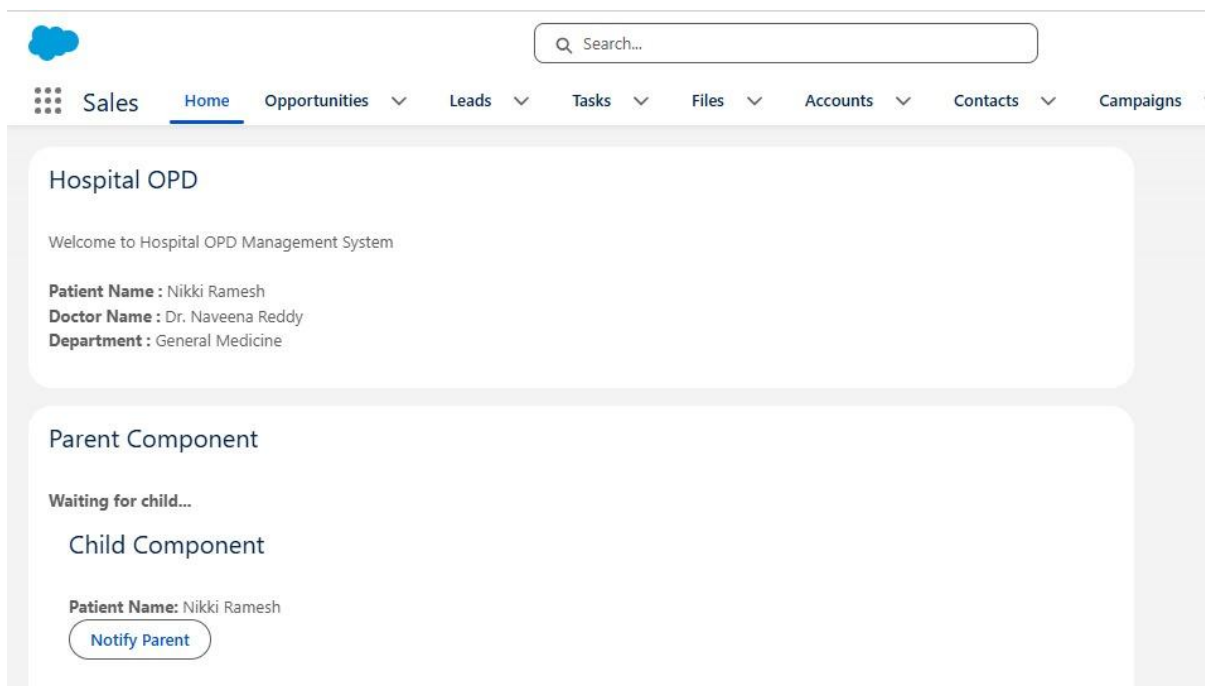
Outcome

The custom Lightning Web Component was successfully deployed and activated using Lightning App Builder. The component was displayed correctly on the Salesforce Home Page, confirming that the deployment process was completed successfully.

Screenshot 2(a) – Lightning App Builder



Screenshot 2(b) – Salesforce Home Page



Task 3: Displaying Hospital Information Using Lightning Web Components

Objective

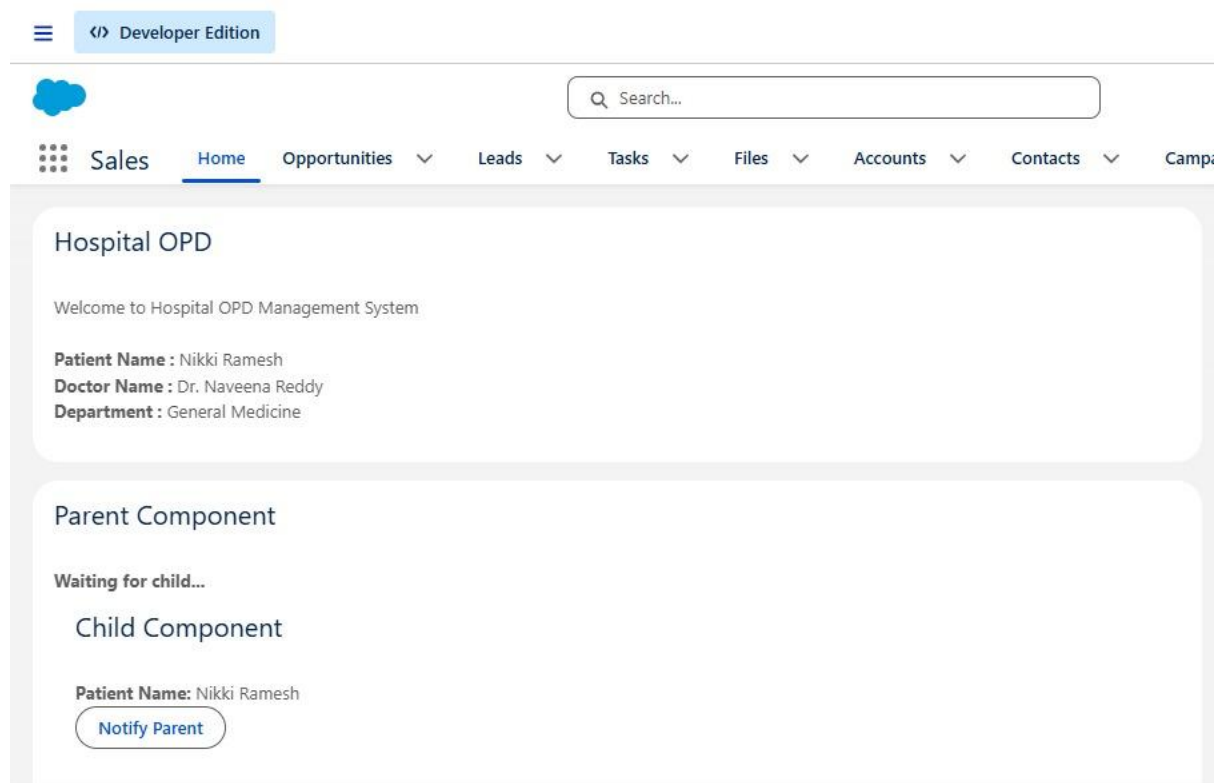
To design an interactive Hospital OPD interface that displays patient information and demonstrates Parent–Child Component communication using Lightning Web Components.

Work Performed

- Declared variables in the JavaScript file to store Hospital OPD information.
- Displayed values such as Patient Name, Doctor Name, and Department on the component page.
- Used LWC template syntax to display variable values dynamically in the HTML file.
- Modified the variable values and verified that the displayed information changed accordingly.
- Confirmed that the component displayed all information correctly after deployment.

Outcome

The Hospital OPD interface successfully displayed all required patient details and demonstrated the Parent–Child Component structure. The component layout was responsive, organized, and suitable for future integration with Apex and Salesforce data.



Task 4: Implementing Button Click Functionality

Objective

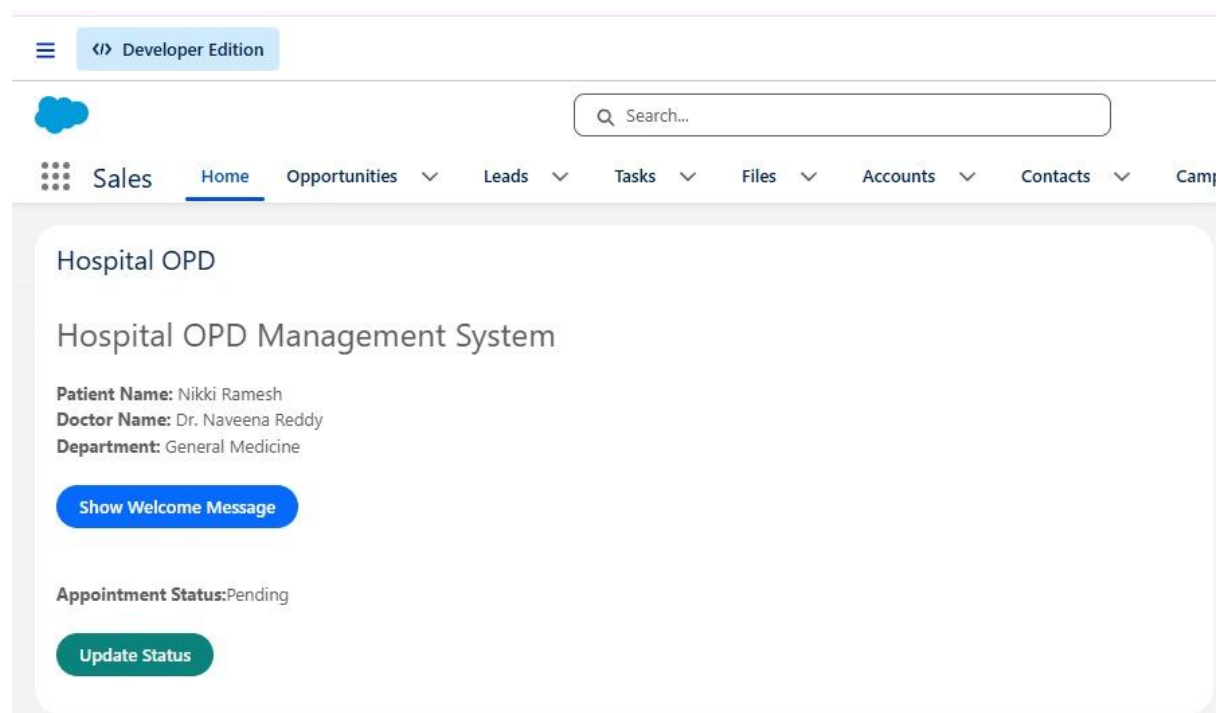
To understand how JavaScript is used in Lightning Web Components by implementing button click events that perform dynamic actions on the user interface without using Apex or database connectivity.

Work Performed

- Enhanced the Hospital OPD Lightning Web Component by adding interactive buttons to improve user experience.
- Created a **Show Welcome Message** button that displays a welcome message when clicked.
- Added an **Update Status** button to dynamically change the appointment status from its initial value to an updated value using JavaScript.
- Implemented event handler methods in the JavaScript file to process button-click events and update the displayed information instantly.
- Used reactive properties in the component to ensure that changes made through JavaScript were immediately reflected on the screen.
- Tested each button multiple times to verify that the expected output was displayed correctly and that the component responded efficiently without requiring any backend processing.

Outcome

The button click functionality was successfully implemented using JavaScript. The component responded instantly to user interactions, displaying the welcome message and updating the appointment status dynamically. This activity demonstrated how Lightning Web Components provide interactive and responsive user interfaces using client-side logic.



Task 5: Developing the Hospital OPD Dashboard

Objective

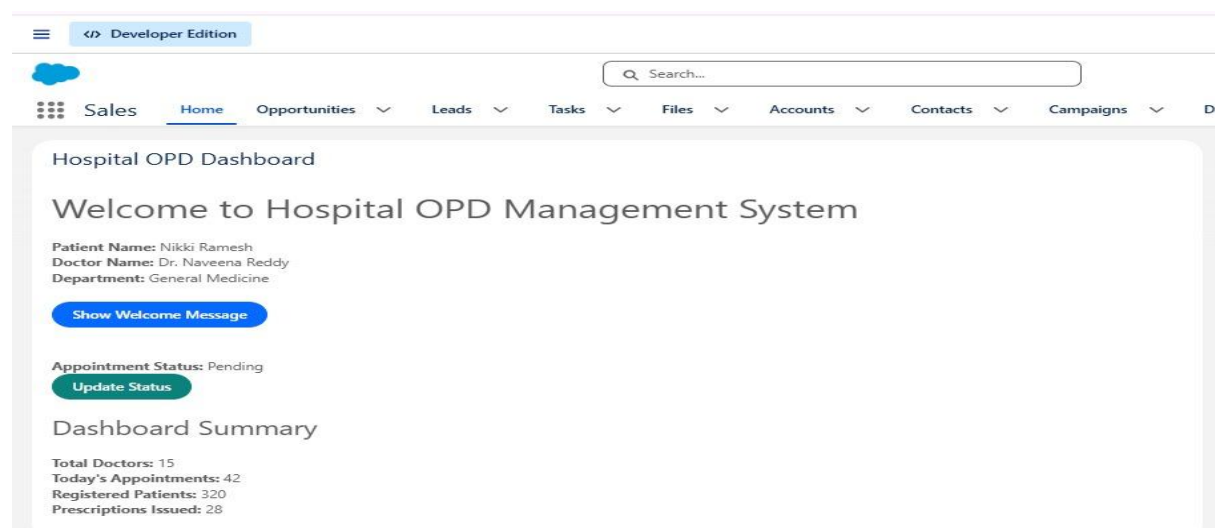
To enhance the Lightning Web Component by creating an informative Hospital OPD Dashboard that displays patient details, appointment information, and hospital statistics in a structured and visually appealing layout.

Work Performed

- Enhanced the existing Hospital OPD component by designing a dashboard-style interface.
- Displayed the Hospital OPD Management System title along with a welcome message to improve the overall presentation.
- Included patient-related information such as Patient Name, Doctor Name, Department, and Appointment Status.
- Integrated interactive buttons for displaying the welcome message and updating the appointment status.
- Created a **Dashboard Summary** section that displayed important hospital statistics including Total Doctors, Today's Appointments, Registered Patients, and Prescriptions Issued.
- Organized all dashboard elements using Lightning Web Component layouts to ensure clarity, readability, and an attractive user interface.
- Verified that all dashboard information and controls were displayed correctly after deployment.

Outcome

The Hospital OPD Dashboard was successfully developed with an organized and interactive interface. It effectively displayed patient information, appointment details, and hospital summary statistics, demonstrating the ability of Lightning Web Components to create professional Salesforce dashboards using static data



Task 6: Understanding and Implementing Data Binding

Objective

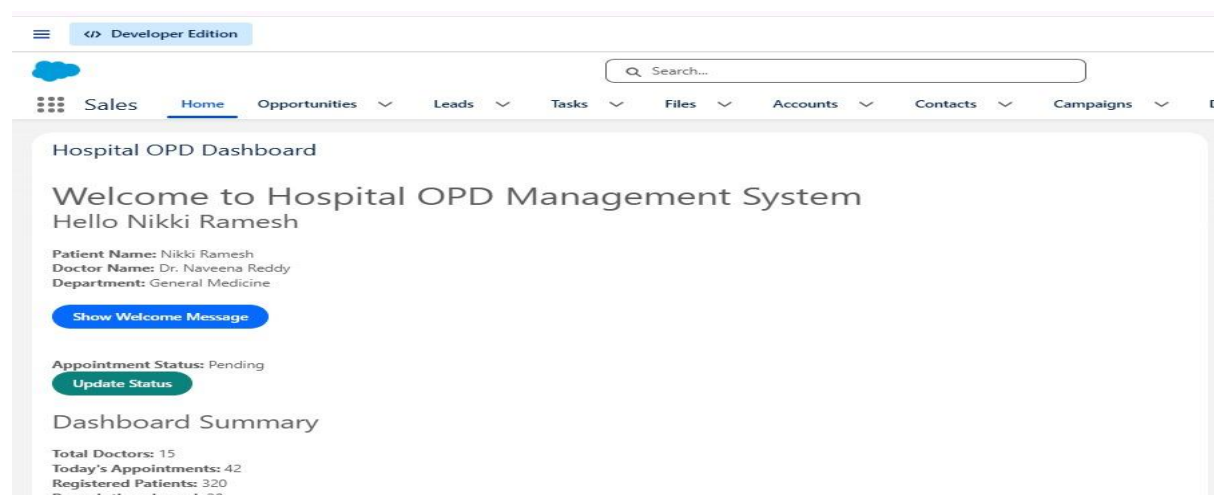
To understand the concept of data binding in Lightning Web Components by displaying dynamic values using JavaScript variables and observing automatic updates in the user interface.

Work Performed

- Declared JavaScript variables to store values that needed to be displayed dynamically in the component.
- Used LWC template expressions to bind the JavaScript variables with HTML elements.
- Displayed a personalized greeting message, **"Hello Nikki Ramesh,"** using the bound variable.
- Modified the variable values and observed that the changes were automatically reflected on the component without manually refreshing the page.
- Verified that the component maintained synchronization between the JavaScript logic and the HTML template, demonstrating the reactive nature of Lightning Web Components.
- Tested the data binding functionality to ensure that all updates appeared correctly whenever the variable values changed.

Outcome

Data binding was successfully implemented in the Lightning Web Component. The application dynamically updated the displayed content whenever the JavaScript variables changed, providing a responsive and user-friendly experience. This activity helped in understanding one of the core concepts of Lightning Web Components and its importance in developing modern Salesforce applications.



Interview Questions

1. What is Lightning Web Components (LWC)?

Answer:

Lightning Web Components (LWC) is Salesforce's modern user interface framework built on web standards such as HTML, JavaScript, and CSS. It enables developers to create fast, reusable, and maintainable components that interact with Salesforce data and provide responsive user interfaces.

2. Why did Salesforce introduce LWC?

Answer:

Salesforce introduced Lightning Web Components to improve application performance, simplify development, and align Salesforce development with modern web standards. LWC offers better performance than Aura Components because it uses native browser capabilities and standard JavaScript instead of a custom framework.

3. What is the difference between LWC and Aura Components?

Answer:

Lightning Web Components (LWC)	Aura Components
Uses standard HTML, JavaScript, and CSS	Uses Aura framework
Faster and lightweight	Comparatively slower
Better performance	Lower performance
Easier to learn and maintain	More complex syntax
Recommended for new development	Mainly used for older applications

4. What are the three files inside an LWC?

Answer:

Every Lightning Web Component consists of three main files:

- **HTML File (.html):** Defines the user interface and page layout.
- **JavaScript File (.js):** Contains the component logic, variables, and event handling.

- **Meta XML File (.js-meta.xml):** Makes the component available in Salesforce and specifies where it can be used, such as Home Pages, App Pages, and Record Pages.
-

5. Why is JavaScript required in LWC?

Answer:

JavaScript is required to implement the business logic of the component. It is used to declare variables, process button-click events, perform calculations, update data dynamically, and control the behavior of the user interface.

6. What is Data Binding?

Answer:

Data Binding is the process of connecting JavaScript variables with HTML elements. When the value of a JavaScript variable changes, the user interface automatically updates without requiring a page refresh, making the application interactive and responsive.

7. Can LWC directly execute SOQL?

Answer:

No. Lightning Web Components cannot execute SOQL queries directly. To retrieve or modify Salesforce data, LWC communicates with Apex classes, which execute SOQL queries and return the required data to the component.

8. Why does LWC need Apex?

Answer:

LWC uses Apex to perform server-side operations such as executing SOQL queries, processing business logic, retrieving records from Salesforce, and updating database information securely.

9. Where is the component deployed?

Answer:

A Lightning Web Component can be deployed to various Salesforce Lightning pages, including:

- Lightning Home Page
- Lightning App Page

- Lightning Record Page

Deployment is done using **Lightning App Builder**.

10. Explain the component you built today.

Answer:

I developed a **Hospital OPD Management System** using Lightning Web Components. The component displays patient information such as Patient Name, Doctor Name, Department, and Appointment Status. It includes interactive buttons for displaying a welcome message and updating the appointment status. I also created a dashboard summary showing Total Doctors, Today's Appointments, Registered Patients, and Prescriptions Issued. The project demonstrated component deployment, JavaScript event handling, and data binding using Lightning Web Components.

Conclusion

The Day 4 provided hands-on experience in developing Lightning Web Components (LWC). A custom Hospital OPD component was created, deployed, and enhanced using JavaScript, data binding, and interactive UI elements. This session strengthened practical knowledge of Salesforce front-end development and improved understanding of LWC concepts.